

第三章 以太坊

本章内容

1.1 以太坊简介

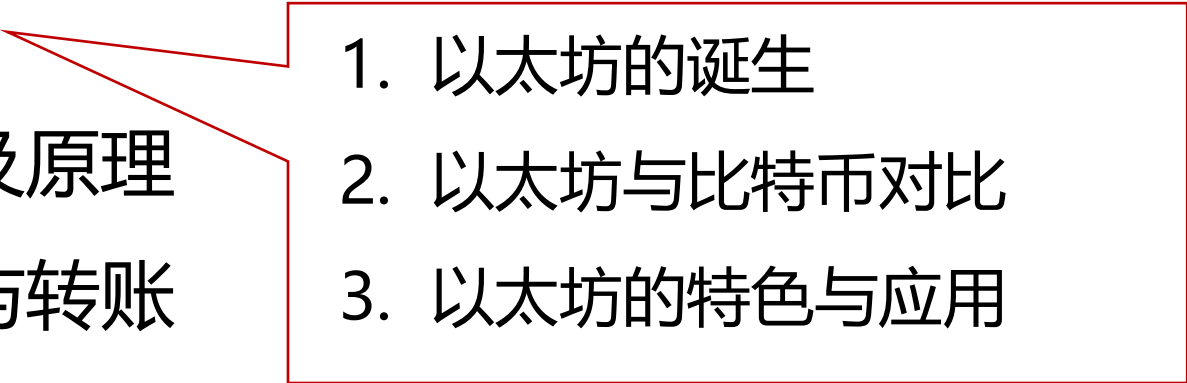
1.2 以太坊基本架构及原理

1.3 以太坊账户模型与转账

1.4 智能合约

1.5 以太坊交易

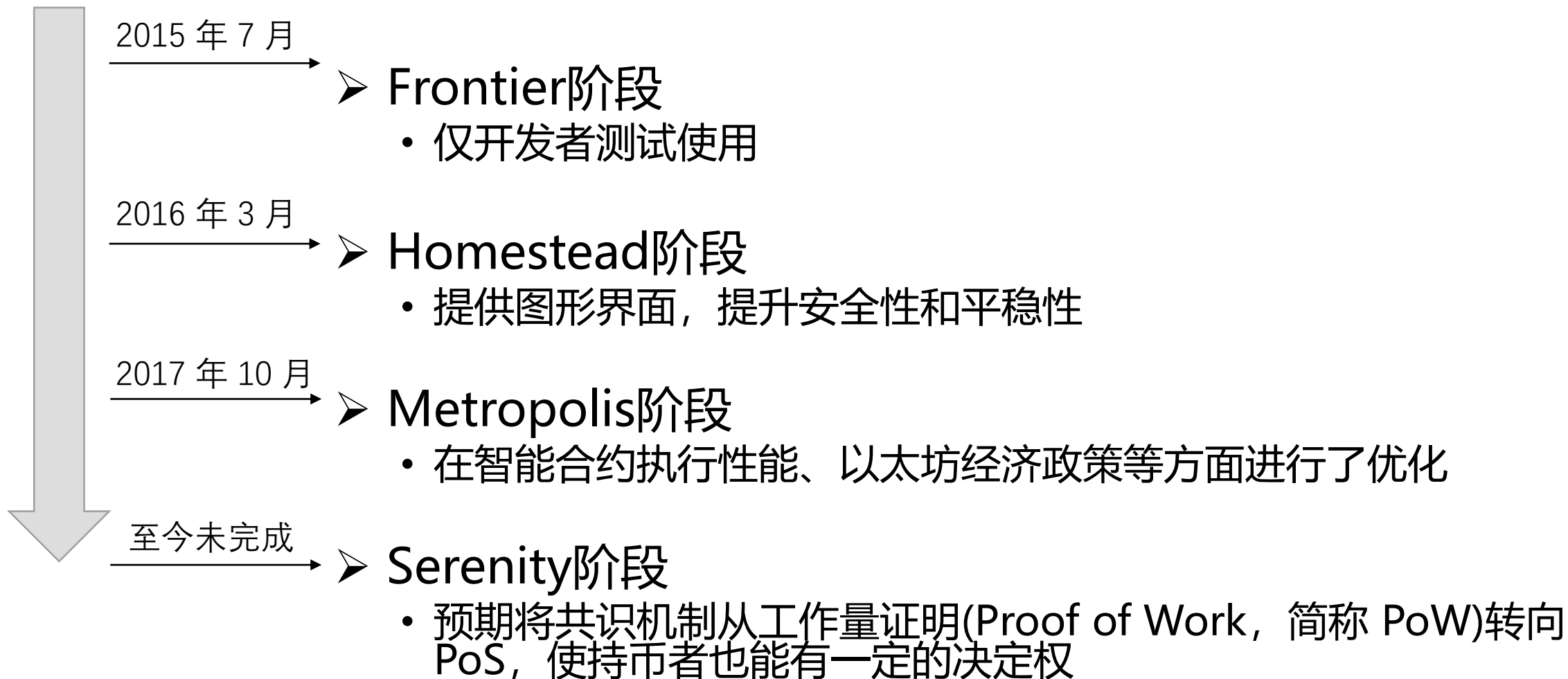
1.6 以太坊数据结构及存储

- 
1. 以太坊的诞生
 2. 以太坊与比特币对比
 3. 以太坊的特色与应用

以太坊(Ethereum)简介

- 第一个支持智能合约的区块链系统
- 使用Ether作为加密数字货币
- 生态社区最活跃的区块链系统
- 超过30,000个github开源项目
- 超过 3000 个基于以太坊的 Dapp
- ...

以太坊的诞生与发展



以太坊与比特币对比

➤ 技术

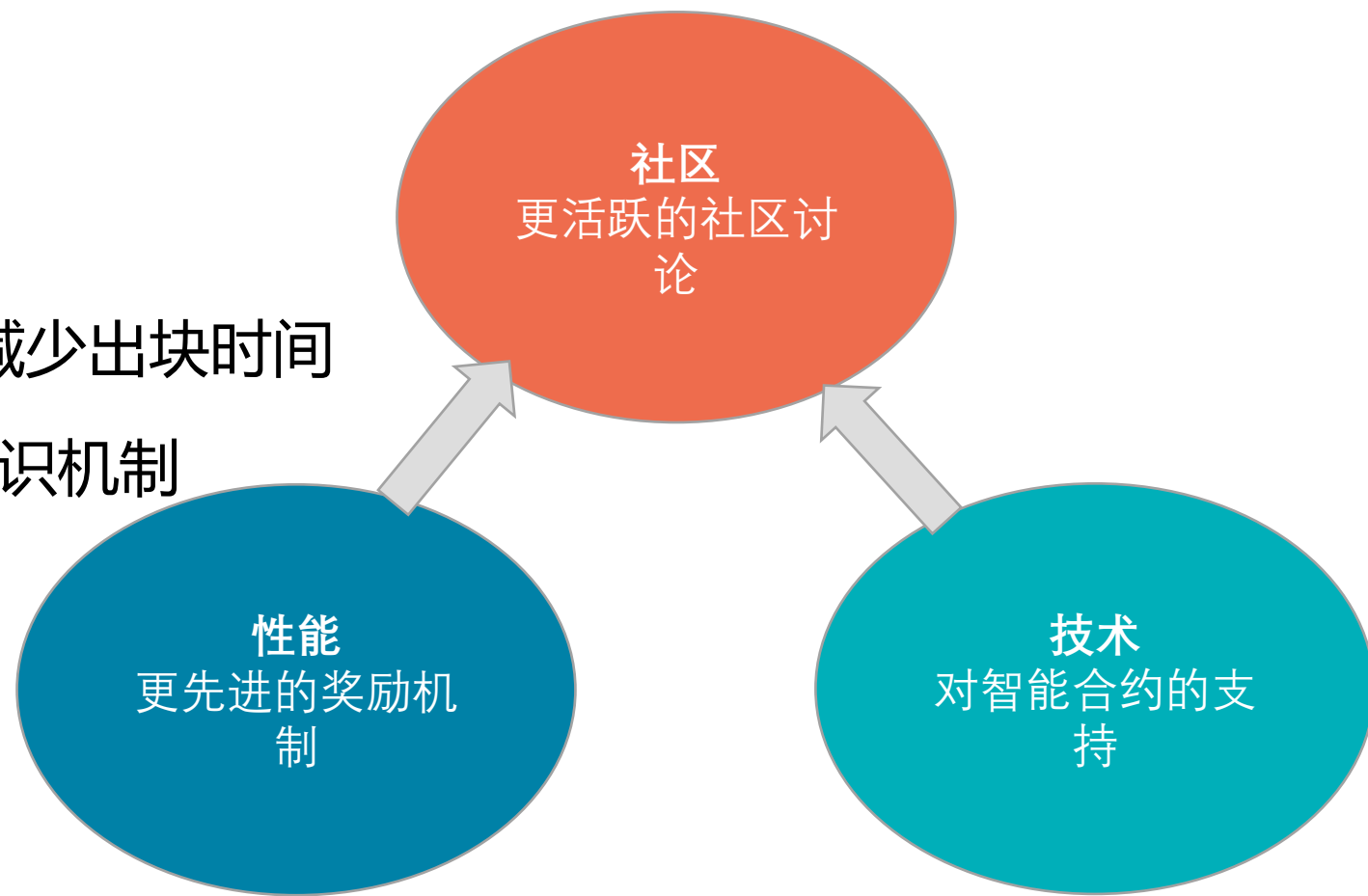
- 支持智能合约

➤ 性能

- 增加叔块(Heaviest)奖励, 减少出块时间
- EtHash (Proof of Work) 共识机制

➤ 社区

- 更加活跃的社区
- 超过35k的开源项目



以太坊特色与应用

- 溯源存证
- 数字资产发行和流通
- 数据共享
- ...

表 3.1 以太坊当前最流行的十个区块链应用

名称	类别	功能描述
MakerDAO	金融	去中心化借贷应用
Chainlink	安全	去中心化预言机
KyberNetwork	交换	代币交换协议
Status	钱包	DApp 浏览器发行的代币
My Crypto Heroes	游戏	角色扮演游戏
Uniswap	交换	代币交换协议
Axie Infinity	游戏	宠物养成游戏
Synthetix	金融	去中心化合成资产平台
Basic Attention Token	钱包	Brave 浏览器的激励代币
Knight Story	游戏	角色扮演游戏

本章内容

1.1 以太坊简介

1.2 以太坊基本架构及原理

1.3 以太坊账户模型与转账

1.4 智能合约

1.5 以太坊交易

1.6 以太坊数据结构及存储

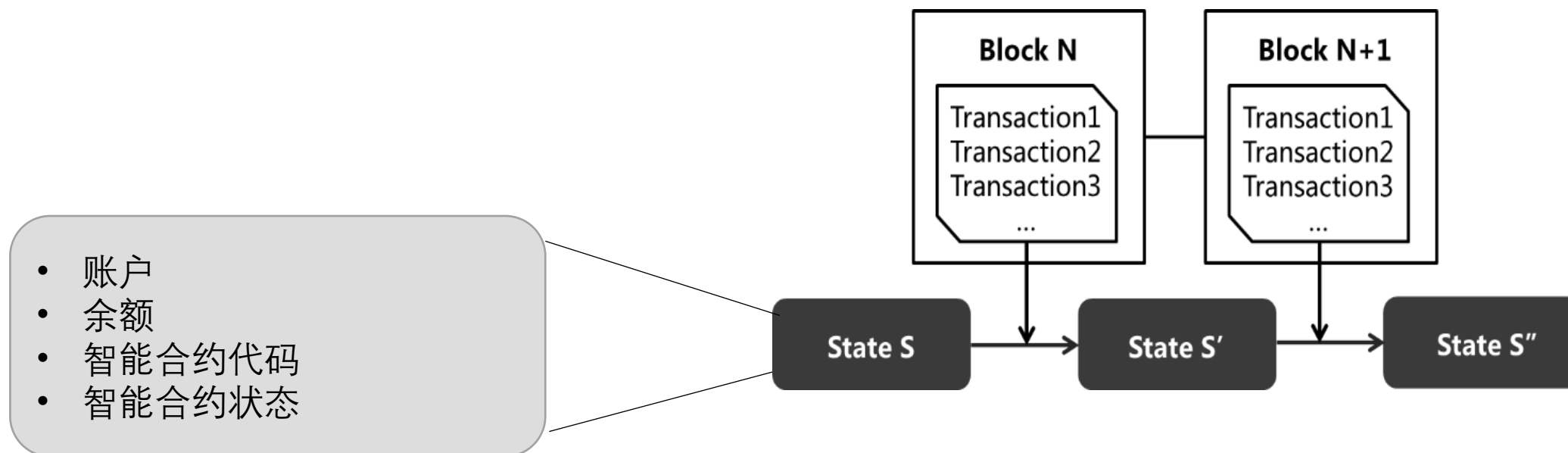
以太坊基本架构及原理

➤ 定义

- 以太坊状态
 - 所有以太坊账户
 - 账户余额
 - 智能合约代码
 - 智能合约状态
- 状态转移
 - 在区块N执行前状态为S，经过区块N的交易进行状态转换后，转换为状态S'，再经过区块N+1的转换后，转换为状态S''

以太坊基本架构及原理

- 以太坊状态转移
 - 智能合约：作用于该状态机转换的代码
 - 以太坊虚拟机（Ethereum Virtual Machine，简称EVM）：执行状态转换代码的虚拟机



以太坊中的状态转移模型示意图

以太坊基本架构及原理

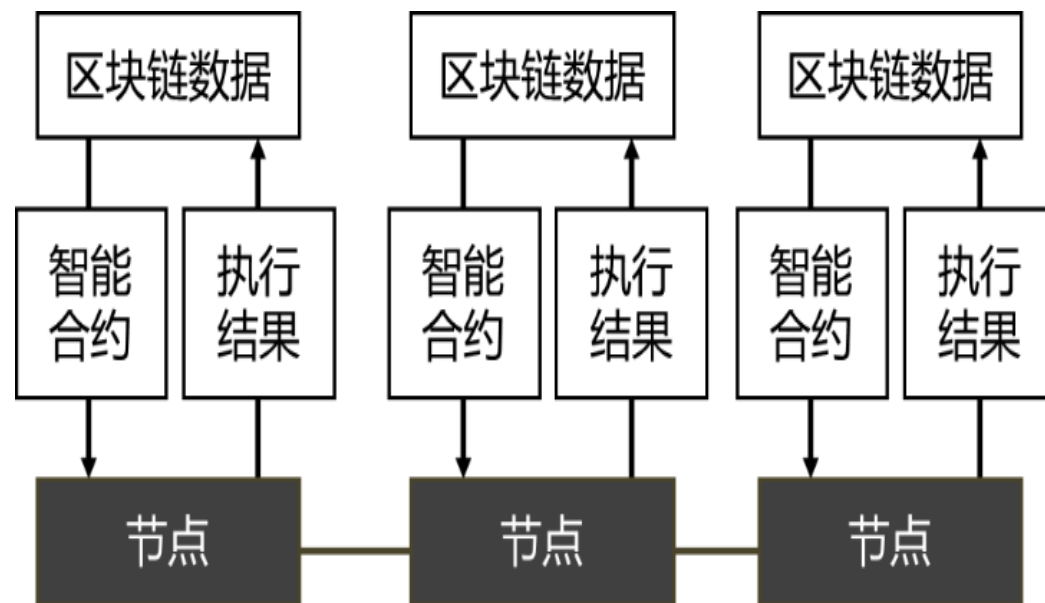
➤ 以太坊状态转移模型的共识问题

- 相比比特币的UTXO模型，状态转换模型虽然使得智能合约的各种变量存储、传参等变得更加灵活，但也带来了多方共识上的困难，如发生分叉时的处理
- 在以太坊的状态转换模型中，如果发生分叉，需要回到分叉时的状态，重新验证另一条分支上的区块
- 以太坊状态的存储采用了一种独立于区块链存在的树形结构，该状态树结构的树根登记在区块中，记为stateRoot，从而使得状态能够在全网得到共识确认，并在分叉时能够快速回滚。

以太坊简易架构图

➤ 定义

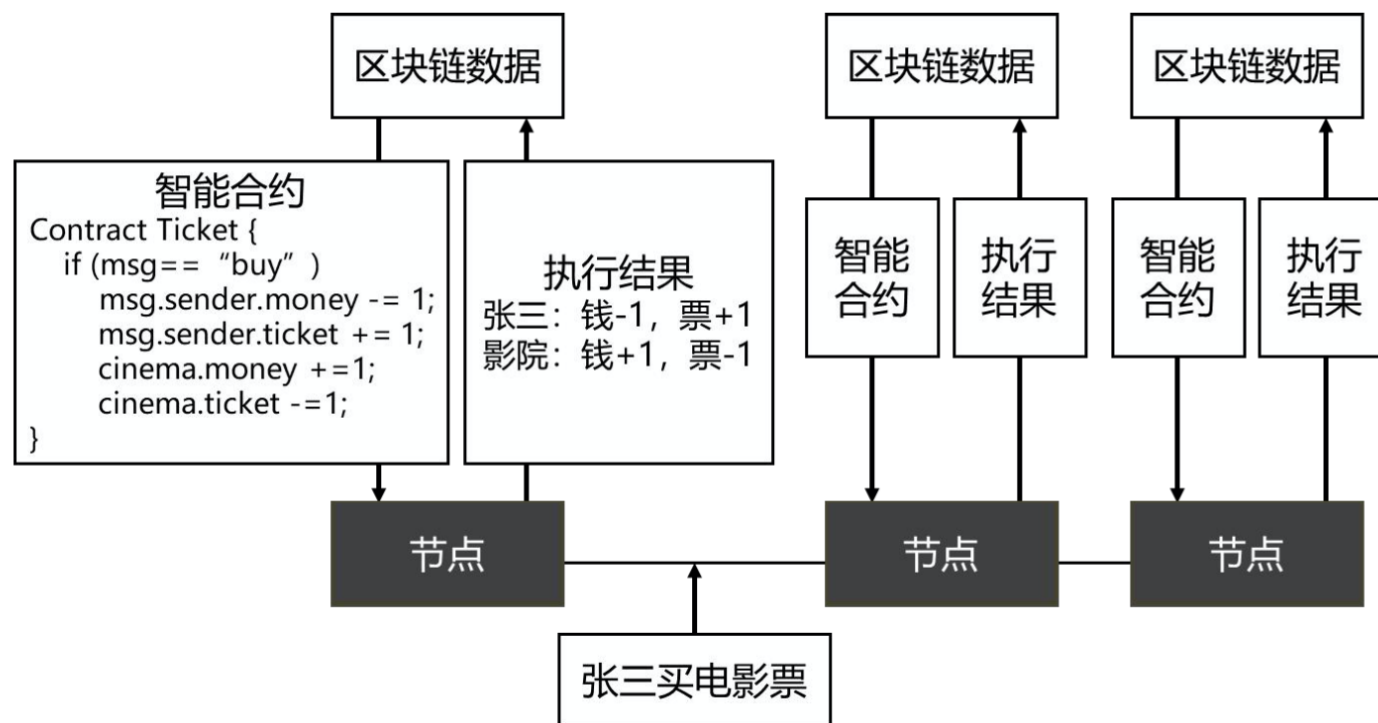
- 区块链数据
 - 以太坊区块链，状态数据，收据数据等
- 智能合约
 - 存储在以太坊上的一段代码
- 节点
 - 保存有区块链数据的节点
 - 每个节点独立的维护区块链数据
 - 节点间采用共识机制达成共识，维护全网的状态一致



以太坊基本原理及例子

➤ 交易执行概述

1. 每个节点独立维护数据
2. 节点独立地在EVM中执行合约
3. 将执行结果写回区块链数据
4. 节点之间执行共识机制，达成共识



“张三买电影票”交易案例

以太坊基本原理及例子

➤ 实例：交易执行具体过程

1. 每个节点独立维护数据，这些节点可以是张三、李四、赵四、影院主管等人
2. “张三买电影票”的交易在所有节点中被独立验证
3. 以太坊虚拟机的执行结果将以某种方式写回到区块链数据中，比如张三的余额、影院的余票等。每个区块中会保留一段摘要，这段摘要为执行完区块中交易后以太坊状态的stateRoot，任一子状态的不同都将导致stateRoot的不同
4. 如果张三控制的节点受到非法攻击或篡改，则执行结果及区块链数据将与网络中其他节点（如李四、赵四、影院的节点）不符，无法参与到网络的下一步共识中

本章内容

1.1 以太坊简介

1.2 以太坊基本架构及原理

1.3 以太坊账户模型与转账

1.4 智能合约

1.5 以太坊交易

1.6 以太坊数据结构及存储

1. 账户模型

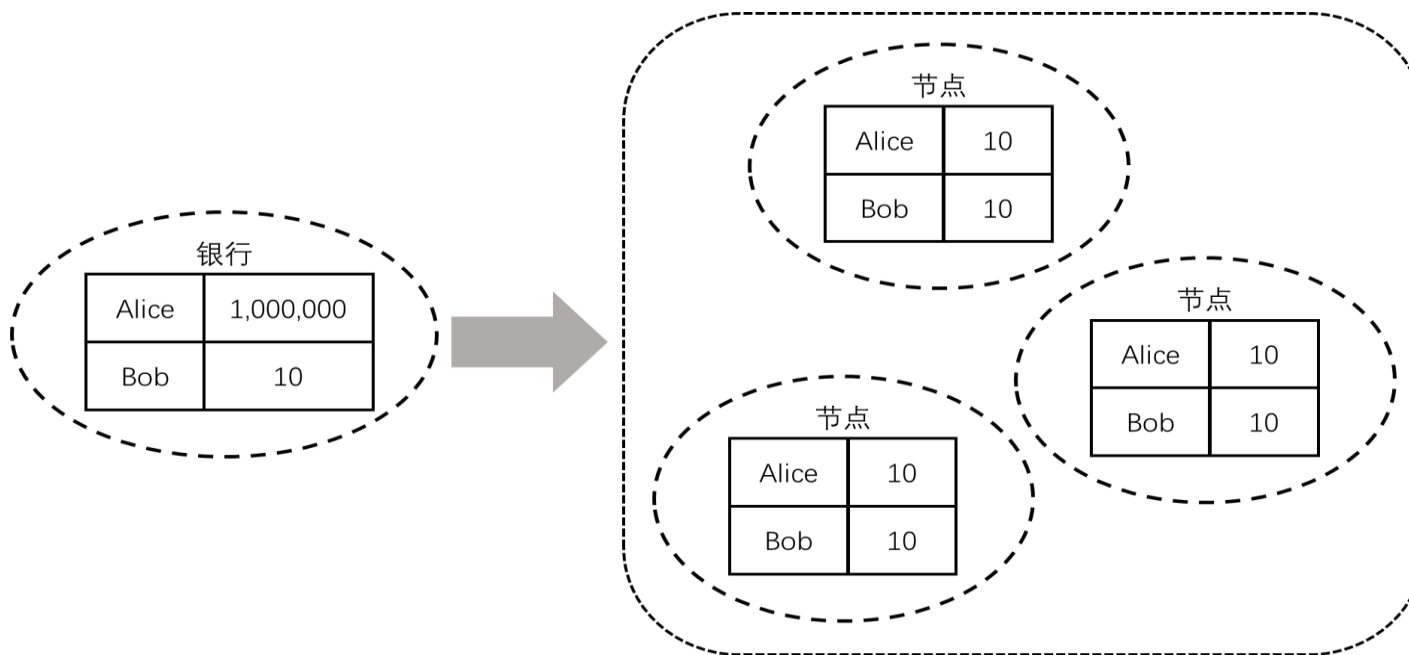
2. 转账

3. Nonce

以太坊账户模型

➤ 背景

- 地址与账户：银行系统与以太坊系统



以太坊的账户模型

以太坊账户模型

➤ 地址的生成

- 计算椭圆曲线下的私钥与公钥

```
7C7FC7CCECC1FEE3B3E4AC63ADD864BD786696C20CA15683A269AA9AD8639443  
(私钥)  
FBD0A98CEF180CDAA3FC2B64A4CA56FBC8E8AF5CC74E820DA34EB9DAB0D46FF2  
E3858707929D0A9BEACF5533C22BF02D34E0A6EE4F1A19C1CEAC4FC4251810A6  
(公钥)
```

- 对公钥使用KECCAK256哈希算法，计算得到一个64位的16进制哈希值（256个比特长）

```
DDC0D707E349E9B726925710238661F085A338F04B0C7C956A796B57018151F0
```

- 截取这个哈希值的最后40位作为一个以太坊地址。

```
238661F085A338F04B0C7C956A796B57018151F0
```


以太坊账户模型

➤ 账户结构

- 用户账户结构保存了用户地址对应账户的数据信息
- **余额**：记录了当前地址持有的以太币的数额，单位是Wei
- **Nonce**：记录了这个地址创建以来累计发起的交易次数

Alice的以太坊地址

0x238661F085A338F04B0C7C956A796B57018151F0

账户余额	1,000,000
累计发起交易	0

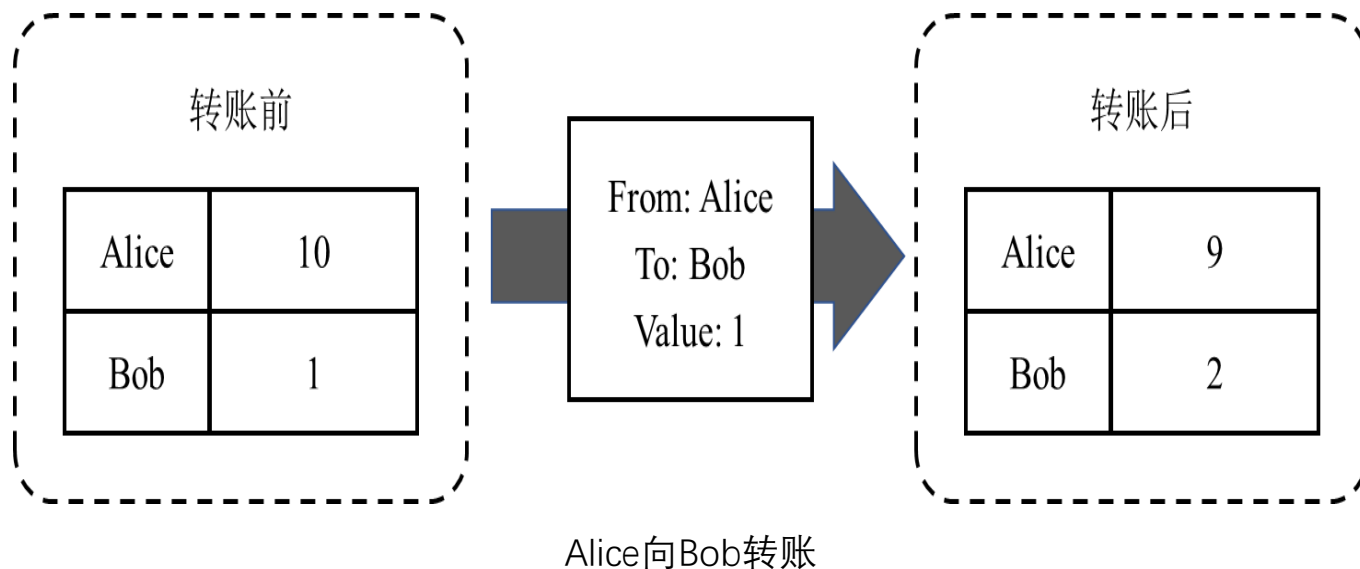
对应

以太坊账户结构

转账

➤ 实例：以太坊的转账交易过程

- 交易的数据通过Alice的私钥进行签名之后得到对应的数字签名
- 以太坊节点通过交易本身和对应的数字签名还原得到Alice的公钥值
- 通过以太坊的地址生成流程，根据Alice的公钥计算出对应的Alice的以太坊地址
- 如果交易受到篡改，计算通过交易哈希和签名计算得到的地址便是错误的



Nonce

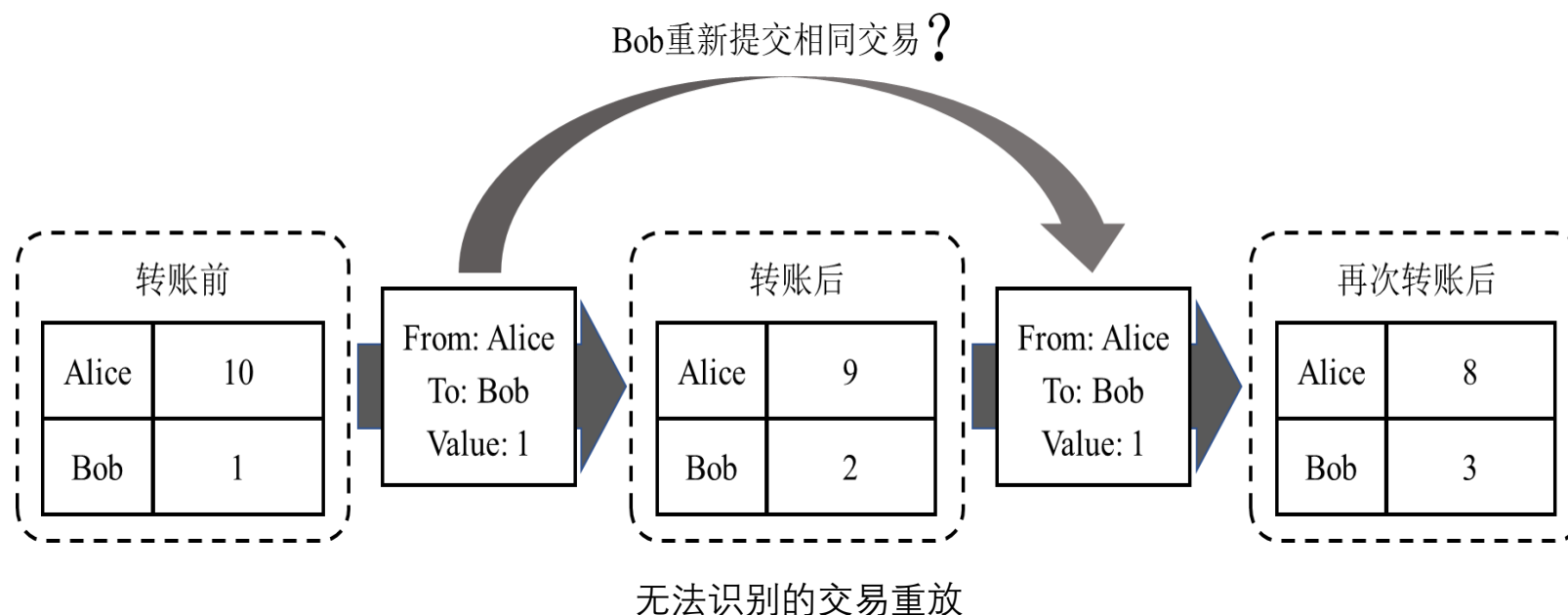
➤ 交易重放问题

- 在以太坊的模型中，交易的合法性检验在于转账发起者的账户余额
- 如果没有其他手段来使得发起过的交易失效，那么这个交易将可以被无限次的重新发起而不需要发起者的同意，因为发起者的签名对于交易始终是有有效的。

Nonce

➤ 交易重放问题

- 如何判断这笔交易是Alice继续向Bob转账，还是在没有Alice同意的情况下进行的一次恶意的攻击行为？



Nonce

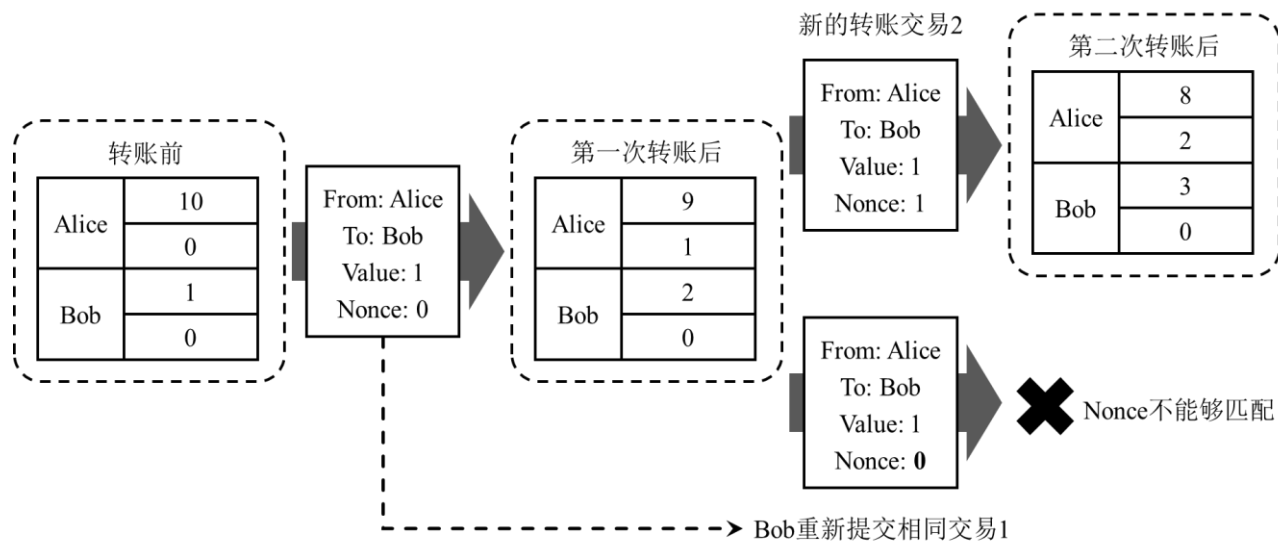
➤ Nonce计数器：计算交易次数和序列

- 只有账户的Nonce和交易的Nonce能够对应的情况下，交易才是合法的
- 当一个交易执行完毕之后，账户的Nonce值增加，原本执行完毕的交易中的Nonce值就无法与现在账户的Nonce值匹配
- 修改对应的Nonce值，意味着原有交易的签名失效，需要发起者的重新签名

Nonce

➤ 实例：Nonce计数器

- Alice向Bob转账之前，Alice的Nonce值为0
- 当Alice向Bob转账1 ETH后，Alice的Nonce值变成1
- Nonce值为1并且具有合法签名的交易2将会是一个合法的交易
- 对于重新提交的交易1而言，此时的交易记录的Nonce值依旧为0



Nonce值防止重复交易

Nonce

➤ Nonce的其他用途

- 控制账户发起的交易的顺序，从而实现一些相对复杂的功能。
- 通过重复提交一个相同Nonce值的交易来使得一个已经提交但是尚未被确认的交易变得不合法，从而实现一定程度的撤销功能。
- ...

本章内容

1.1 以太坊简介

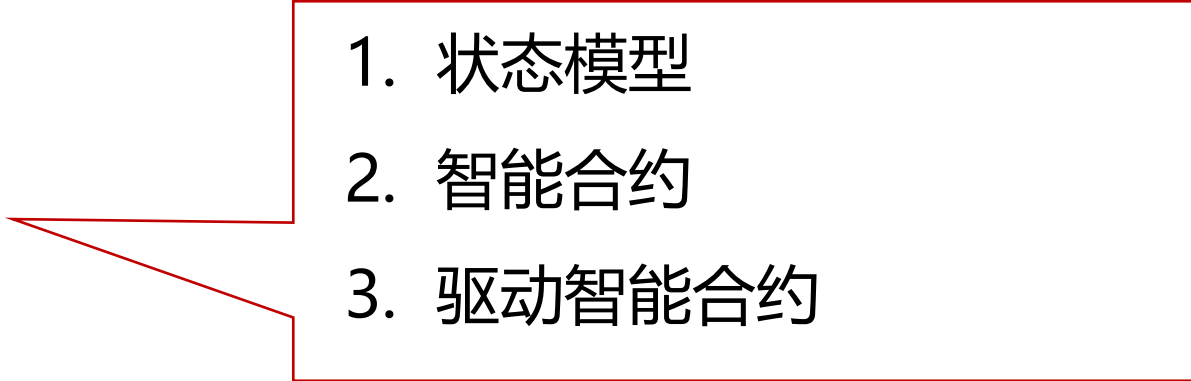
1.2 以太坊基本架构及原理

1.3 以太坊账户模型与转账

1.4 智能合约

1.5 以太坊交易

1.6 以太坊数据结构及存储

- 
1. 状态模型
 2. 智能合约
 3. 驱动智能合约

智能合约

➤ 定义：智能合约

- 一段在区块链上执行的代码，它依托于区块链系统在参与者之间实现对执行的一致认可

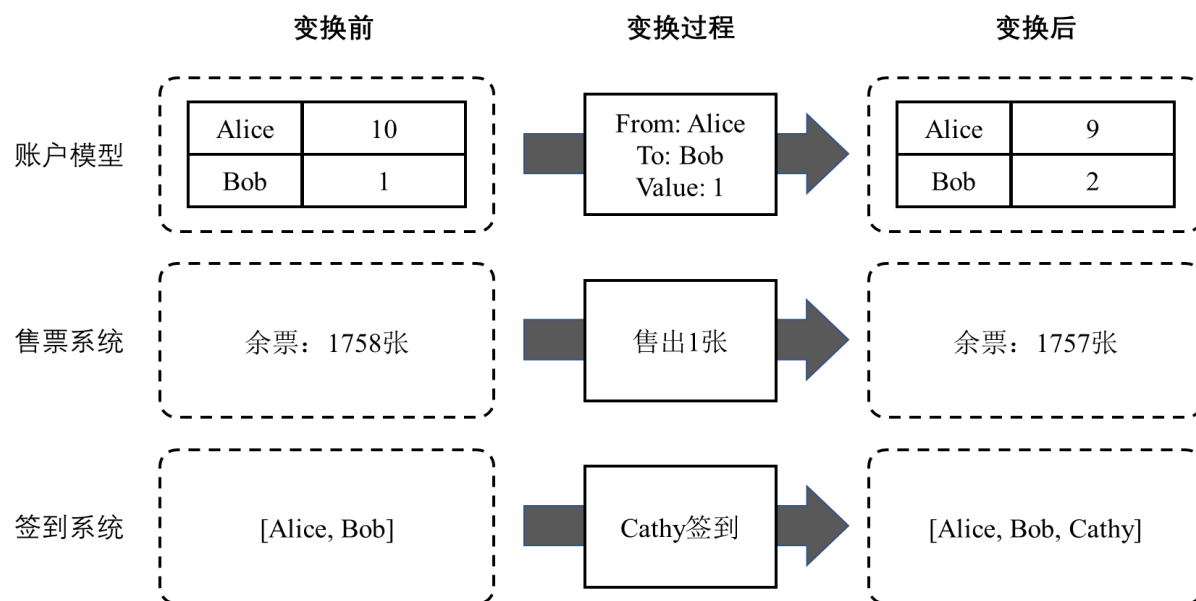
状态模型

➤ 背景

- 在账户模型中，用户的余额通过地址上的账户数据来表示，具体为账户数据结构中的一个余额的数值
- 在转账交易的过程中，通过转账预先定义好的语义，在发起者的账户中减去交易中定义好的转账金额，在接受者的账户中增加相应的金额
- 可以把账户的余额泛化成一种**账户的状态**，而把转账交易当作是**改变状态的一个方法**

状态模型

- 只要约定好的变换规则是固定的，就可以不断地通过变换规则不停地作用到原有状态上产生新的状态
- 对于有着相同的初始状态的参与者们来说，经过完全相同的变换过程，最后必然得到完全相同的当前状态。



三种系统下的状态变换例子

智能合约

➤ 状态模型上的代码执行

- 把计算机程序执行过程中的变量等数据一同存储到区块链上，交易的过程则是执行一段约定好的计算机程序
- 在整个流程中，这些存储的数据是一个状态，执行的代码是一种特定的变换
- 执行的过程是一个单射，对于当前的状态，不能得到多种结果

智能合约

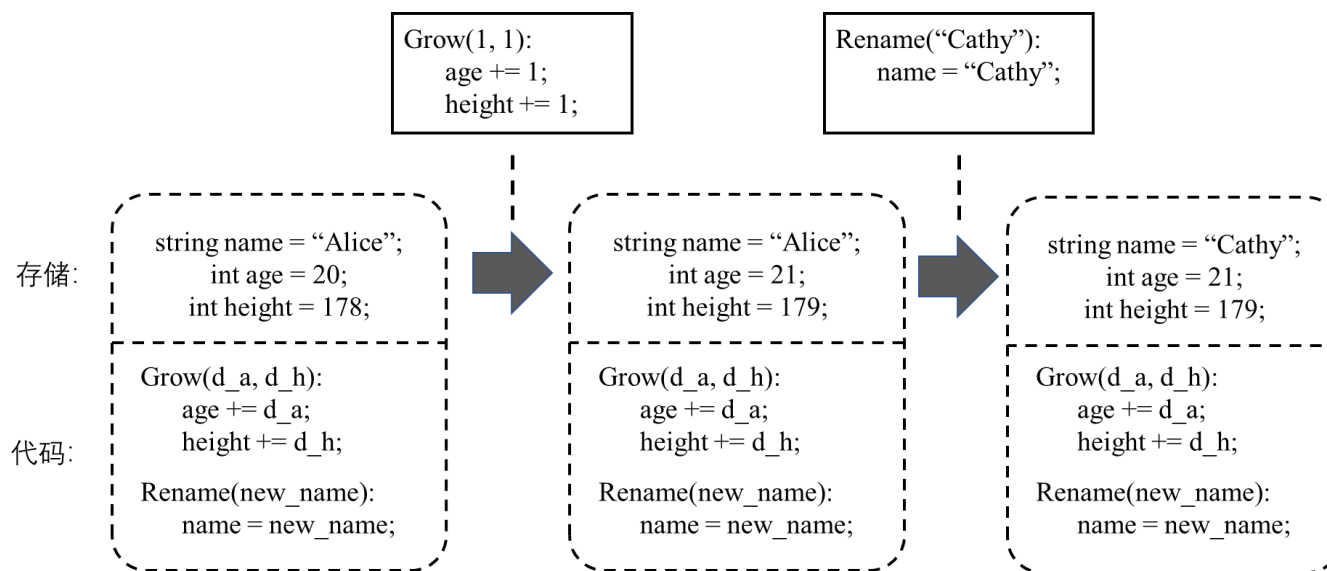
➤ 智能合约的代码执行

- 利用提前约定好的代码来管理和变化存储在以太坊上的状态变量
- 利用智能合约的代码来自定义交易过程中的状态变换过程
- 在可以受到以太坊系统的参与者一致认可的条件下不断执行和变化，实现“世界状态机”

智能合约

➤ 实例：记录Alice个人信息的智能合约

- 合约使用状态模型保存个人信息 <name: "Alice" , age: 20, height: 178>
- 制定Grow和Rename两种方法来进行状态修改

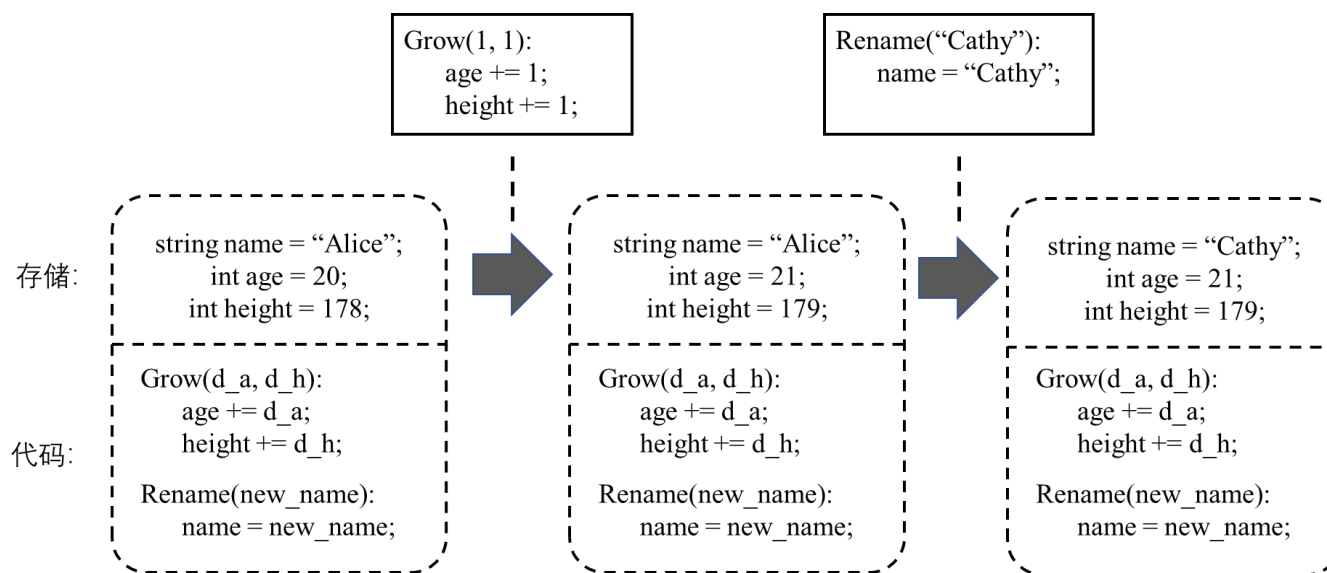


记录Alice个人信息的合约

智能合约

➤ 实例：记录Alice个人信息的智能合约

- 状态转移过程一：执行Grow (1, 1)
- 状态转移过程二：执行Rename ("Cathy")
- 状态变换结果：<name: "Cathy" , age: 21, height: 179>



记录Alice个人信息的合约

合约账户与数据存储

➤ 背景

- 对于智能合约与状态的实体，以太坊同样使用了账户作为一个合约实例对象的抽象，它基本上是普通账户的扩展

合约账户与数据存储

➤ 定义

- 合约账户

- 表示合约实例对象的账户被称为合约账户，它的地址并不是由公钥产生，而是通过特定的算法在创建合约的时候生成

- 外部账户

- 表示用户余额的账户则被称之为外部账户（Externally Owned Accounts），这些账户受外部私钥控制，由外部的用户所操控。这里的外部是相对于以太坊运行机制而言的

合约账户与数据存储

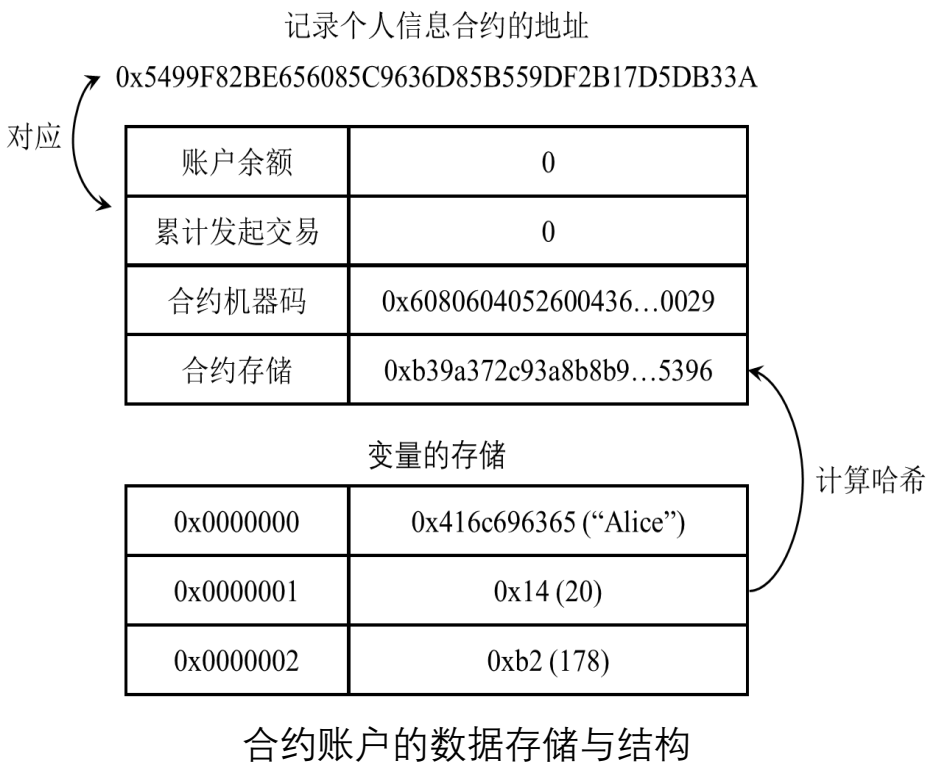
➤ 合约账户内容

- 数据结构
 - 合约账户的数据结构基于原本外部合约账户，主要扩展了存储合约代码和存储合约状态的字段
- 计算机代码
 - 合约的计算机代码通过机器码的形式保存在合约机器码的字段中
- 状态存储
 - 合约的状态存储保存在一个存储的映射表之中，账户的内部只保留了整个存储表的哈希值
- 合约账户不由具体公钥和私钥进行控制，不能够从合约地址发起任何以太坊的交易，所以在绝大多数情况下合约的Nonce值不会改变

合约账户与数据存储

➤ 实例：记录Alice信息合约的数据存储

- 合约操作
 - Grow和Rename操作以机器码的形式保存
- 合约变量
 - 变量存储
 - 姓名保存在变量存储表0x0000000
 - 年龄保存在变量存储表0x0000001
 - 身高保存在变量存储表0x00000002
 - 变量的哈希值
 - 对整个变量存储表计算得到哈希值
0xb39a372c93a8b8b970e359a978fba643f94ac966c0d862e27da777
0d8f485396，保存在合约账户中



合约地址的生成

➤ 背景：

- 合约地址并不取决于外部的公钥，而是通过特定的算法计算得到。在以太坊中提供了两种生成合约地址的方法

➤ 合约地址生成方法

- 通过合约创建者的地址和Nonce计算得到
- 通过合约创建者地址、指定的初始化值和合约代码的哈希值计算得到

合约地址的生成

➤ 实例：通过合约创建者的地址和Nonce生成合约地址

- 输入：

- 创建者地址： `0x238661F085A338F04B0C7C956A796B57018151F0`

- Nonce值： `0`

- 使用以太坊自定义的RLP (Recursive Length Prefix) 编码格式序列化数据

```
0XD694238661F085A338F04B0C7C956A796B57018151F080
```

- 将序列化数据通过SHA256计算，得到长度为256位的哈希，值，取最后的160位

```
0x5499F82BE656085c9636d85b559df2B17d5db33A
```

合约地址的生成

- 实例：通过合约创建者的地址和Nonce生成合约地址
 - 创建合约的时候需要使用到Nonce值
 - 当出现合约创建合约的情况的时候，创建其他合约的合约账户的Nonce值便需要改变，否则该合约账户每一次计算得到的新合约地址都相同

驱动智能合约

➤ 背景

- 在状态模型的框架下，以太坊的状态通过交易来改变，智能合约的状态变化同样使用交易来实现。
- 在整个合约的生命周期中，所有的状态变换都是通过执行特定交易来实现的，智能合约的每次运行都是通过交易来驱动。

调用合约

➤ 背景

- 在一次交易中，以太坊按照事先的约定执行智能合约的代码，最后得到运行的结果

➤ 定义

- 智能合约作为交易的接收方，按照交易发起者指定的函数和参数进行执行，这些接收者是合约账户地址的交易也被称作合约调用

调用合约

➤ 如何调用合约函数

- 交易接收方
 - 指定智能合约所在地址作为交易接收方
- data字段
 - 为了实现调用过程中指定智能合约的不同函数以及携带函数的参数，以太坊在交易中加入了data字段用于存放这些数据。
- 合约函数索引
 - 在现有的ABI约定中，使用了函数名的哈希值作为调用过程中函数的索引，并在这个哈希值的后面附上经过序列化编码的参数。

调用合约

➤ 实例：智能合约的调用

- 调用信息合约中的Grow(int256, int256) 函数，需要知道
 - 合约账户地址
 - 被调用的合约函数

- 合约账户地址

0x5499F82BE656085c9636d85b559df2B17d5db33A

- 被调用的合约函数

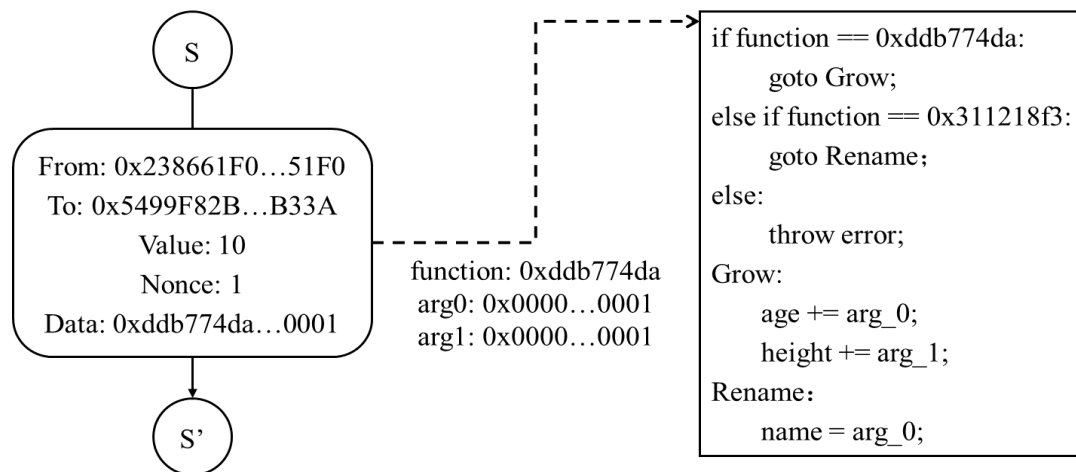
- 计算Grow(int256, int256)的SHA256哈希值并取前64位，得到

0xddb774da

调用合约

➤ 实例：调用智能合约函数Grow(1,1)

- 构造交易
 - 交易接收方：合约账户地址
 - Data字段：0xddbb774da0000...00010000...0001
 - 0xddb774da是要调用的函数
 - 0x0000...0001和0x0000...0001是256位的参数



智能合约的调用

创建合约

➤ 背景

- 智能合约的代码存储在账户之中，账户中的代码从无到有同样也是一种状态的变化

➤ 定义

- 通过发送交易将代码存储到以太坊的合约账户之中，这个过程便是合约的创建，也叫作合约的部署

创建合约

➤ 智能合约的创建问题

- 发送交易的时候，不会有合约账户地址和合约账户
- 发送交易的时候，不会有可以被执行的智能合约代码

创建合约

➤ 以太坊的特殊化处理

- 创建合约的交易没有接收地址，交易中的To字段始终为0。同样地，对于一个接收地址为0的交易，以太坊都会认为是一个创建合约的交易。在检查交易无误之后，以太坊会根据发送者的地址和Nonce值计算出这个交易创建的合约的账户地址
- 交易的data字段不再是作为执行过程中的参数，而是直接运行交易data字段中的内容。创建合约的时候，需要将合约的代码和一些初始化的代码放置到交易的data字段之中，经过运行之后得到合约的初始状态。

停机问题与Gas

➤ 背景

- 一个交易如果调用的智能合约存在死循环，那么这个交易的执行将不会停止，这对于整个以太坊系统来说是一个严重的打击
- 停机问题
 - 对于一个图灵机而言，任何人都不能判断它是不是能够在有限的时间内结束运行

停机问题与Gas

- Gas机制：保证智能合约能够在有限时间内能够终止
 - 以太坊智能合约运行的每一个操作都规定了需要消耗的Gas的数值，并要求交易的发起者预先支付Gas额度
 - 每次运行智能合约代码的时候，每一步操作都会消耗掉一些预先支付的Gas值，直到交易中预支付的Gas额度被消耗殆尽

以太坊虚拟机

➤ 背景

- 智能合约的代码以何种格式存储和运行？
 - 使用高级语言：对于去中心化的网络来说，参与的节点类型可能各有千秋，可能得到不同的执行结果
 - 使用机器码：合约的执行跟特定架构相关
 - 折中方法：使用统一的虚拟架构和机器码

以太坊虚拟机

- 以太坊虚拟机：一个256位的栈虚拟机
 - 256位：
 - 指执行过程中的数据宽度是256位，相比之下普通的x86或者ARM架构通常是32位或者64位；
 - 栈虚拟机：
 - 指EVM的执行流程基于一个栈结构，所有的指令都是操作栈顶的数据

以太坊虚拟机

➤ 以太坊虚拟机指令

- 智能合约的编写
 - 开发者使用类似于Solidity或者Vyper这些上层的高级语言来编写智能合约的代码
- 智能合约的编译
 - 将高级语言编写的智能合约编译成EVM能够识别的机器码指令，从而能够在以太坊的平台上使用EVM执行

本章内容

1.1 以太坊简介

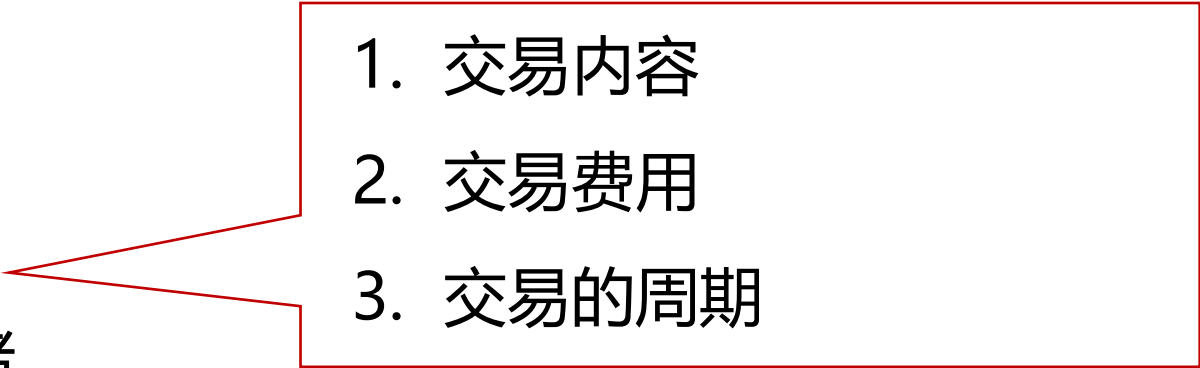
1.2 以太坊基本架构及原理

1.3 以太坊账户模型与转账

1.4 智能合约

1.5 以太坊交易

1.6 以太坊数据结构及存储

- 
1. 交易内容
 2. 交易费用
 3. 交易的周期

交易内容

➤ 背景

- 在以太坊中，交易承载了账户转账和创建、调用合约等功能，数据的内容更为复杂，大体上可以粗略的分为三个部分，即基本的交易，驱动智能合约和交易的签名

交易内容

➤ 基本交易内容

- from: 交易发送者的地址。发送者地址可以通过合约的签名信息 $\langle r, s, v \rangle$ 计算得到，实现上交易的数据结构中并不会存储发送者地址
- to: 交易接收者的地址。在进行转账时是接受转账金额的地址，在创建合约时设置为 `0x0000...000`，在调用合约时则是合约的地址
- value: 交易的金额，单位是Wei。Wei是以太币最小单位，我们常说的1个以太币是单位Ether， $1 \text{ Ether} = 10^{18} \text{ Wei}$

交易内容

➤ 驱动智能合约

- data: 交易附带数据, 传递创建合约的代码和构造函数, 或调用合约的函数及参数
- nonce: 交易发送者累计发出的交易数量, 用于区分一个账户的不同交易及顺序
- gasPrice: 发送者支付给矿工的gas的价格, 用于实现从Gas到以太坊货币单位的转换, 从而计算使用的Gas的总价格
- gasLimit: 该交易允许消耗的最大的gas, 用于解决智能合约不能停机的问题

交易内容

➤ 交易的签名

- hash: 由以上字段生成的哈希值, 也作为交易的id。
- r、s、v: 用于ECDSA验证的参数, 由发送者的私钥对交易的哈希做数字签名生成, 用于确认转账的合法性。

交易费用

➤ 背景

- Gas机制不仅可以保证合约停机，同时可以对交易执行的成本进行归一化计算，以太坊中通过Gas进行计算交易的费用

➤ 主要概念

- Gas：以太坊中资源消耗的基础单位
- GasLimit: 允许消耗的最大Gas值
- GasUsed：执行后消耗的最大Gas值
- GasPrice：用户为消耗的每个Gas单位支付的以太币

交易费用

➤ Gas与GasUsed

- 在交易的执行过程中，每笔交易都带有基础Gas消耗值，用户在创建或调用智能合约的过程中，对以太坊虚拟机的不同操作都将消耗不同值的Gas。
- 基础的交易Gas值加上以太坊虚拟机运行时的Gas消耗值，即构成了交易的GasUsed。

交易费用

➤ GasLimit

- 交易的GasUsed是实时计算的，即以太坊虚拟机的每步操作都将计算累积一次，如果交易的GasUsed超过了用户定义的GasLimit，则判定为Gas不足，交易执行失败。

交易费用

➤ GasPrice

- 交易执行完成后，将得到交易的GasUsed乘上GasPrice，即为用户该笔交易应付的手续费，这一手续费从交易发起账户扣除，加到区块Coinbase账户中。
- 挖到区块的节点除了得到区块奖励外，还将得到运行以太坊智能合约的手续费。同样地，区块中也带有GasLimit和GasUsed字段。

交易费用

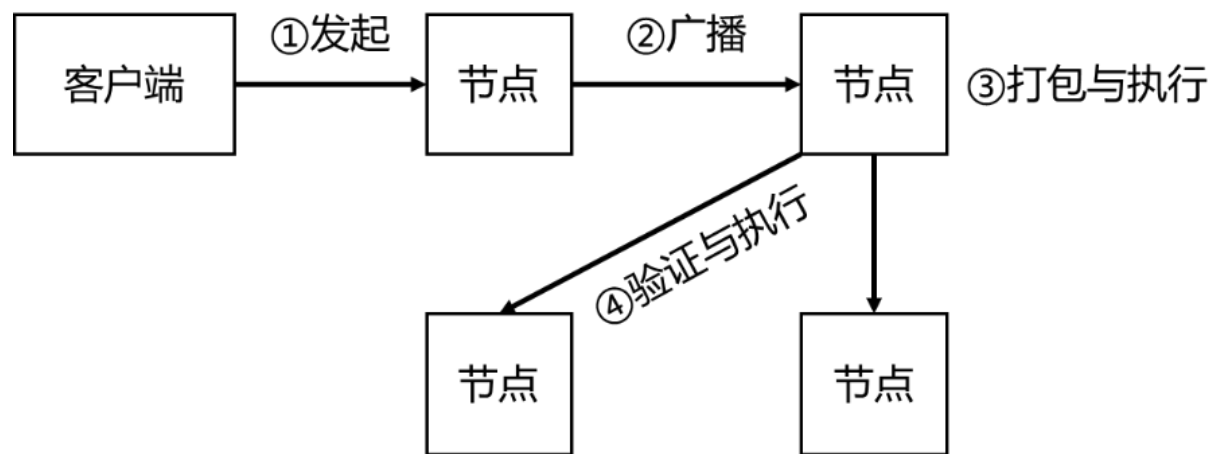
➤ Gas的弊端

- 以太坊虚拟机的每个操作的定价是以太坊社区的开发者决定的，定价的合理性也时常受到质疑。在以太坊两百多万的区块高度上，曾经出现过针对Gas定价不合理的攻击。

交易的周期

➤ 以太坊交易周期

- 发起
- 广播
- 打包与执行
- 验证与执行



以太坊交易周期

发起

➤ 发起

- 设定交易内容
 - 用户在本地的以太坊钱包软件中选择要发送交易的地址 (from) , 输入目标地址 (to) 、金额 (value) 、是否部署或调用合约 (data) 、手续费单价 (gasPrice) 等
- 发起交易
 - 用户确认发送至以太坊节点, 多个用户各自保有钱包私钥, 而通过同一个以太坊节点广播交易
- 以太坊钱包软件补充内容
 - 以太坊钱包软件将自动为用户得出交易所需的燃料上限 (gasLimit) , 并给出用户地址对应的nonce值, 最后使用私钥得到r、s、v, 最终将交易序列化后发送到网络中
 - 部分客户端中, gasLimit与nonce也可以是用户自定义的。

广播

➤ 广播

- 节点收到（或自己发起）交易后，对交易进行验证
- 节点验证交易为合法交易后，将交易加入节点的交易池中
- 节点验证交易通过后，除了加入节点的交易池中，还会根据P2P网络广播的策略向相邻节点继续广播该交易

打包与执行

➤ 打包与执行

- 交易进入内存池后，具有挖矿功能的全节点，开始打包下一个区块
- 节点将交易打包时，对交易进行逐个执行，每笔以太坊交易都是对以太坊状态的修改
- 在所有需要打包的交易执行后，交易、状态以及收据的信息也会打包到区块中
- 记账节点在打包交易并获得合法的区块后，将区块（包含交易数据）广播到网络中的相邻节点

打包与执行

- 交易类型：根据to值的不同，将交易分为3种执行类型
 - 创建合约交易
 - 调用合约交易
 - 普通转账交易

打包与执行

- 创建合约交易： to为空的交易
 - EVM根据from值及nonce值生成合约地址
 - 执行data中对应的智能合约代码
 - 将合约EVM代码存储到合约地址中

打包与执行

- 调用合约交易：to为合约账户的交易
 - EVM将从世界状态中获取to地址中存储的EVM代码
 - to地址中存储的是合约本身
 - 执行交易的data字段中包含的代码
 - data中则包含了调用合约的相应函数及其参数
 - 本质上讲，对合约的调用，是对合约状态的修改。

打包与执行

- 普通转账交易：to为外部账户的交易
 - 直接将以太币金额从from转到to。

验证与执行

➤ 验证与执行

- 没有获得记账权的节点，在收到广播的区块后，对区块进行合法性的验证，并进行交易的执行
- 验证的内容与执行的过程与2、3中的相同，保证智能合约执行的去中心化

本章内容

1.1 以太坊简介

1.2 以太坊基本架构及原理

1.3 以太坊账户模型与转账

1.4 智能合约

1.5 以太坊交易

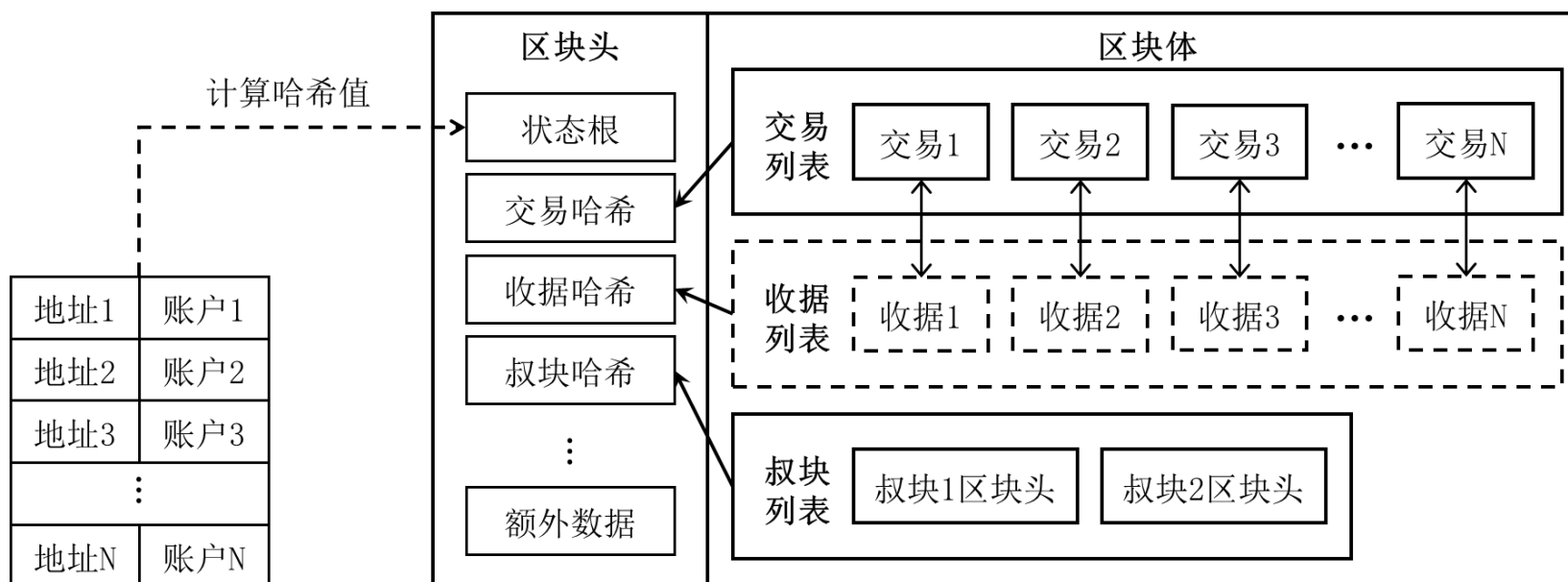
1.6 数据结构及存储

- 
1. 区块与叔块
 2. Merkle Patricia Trie
 3. 布隆过滤器

区块与叔块

➤ 背景

- 以太坊的区块同样是打包成一批执行的交易，它的数据结构同样分成了区块头和区块体。



区块与叔块

➤ 以太坊区块头内容

- 记录以太坊状态的状态根
- 交易列表、收据列表和叔块列表对应的哈希值
- 最长不超过100KB的额外数据

➤ 以太坊区块体内容

- 交易组成的交易列表
- 由交易执行信息组成的收据列表
- 用于改进以太坊共识过程的叔块列表

世界状态

➤ 背景

- 由于以太坊使用了状态模型，在区块中除了保存交易列表之外，还附带了当前区块中交易执行结束状态相关的信息

➤ 定义

- 在以太坊中账户内部存在着各自的状态，他们在特定交易下发生变化，将以太坊中所有存在的账户的状态全部汇总到一起，就可以得到一个全局的状态，它也被称作“世界状态”

➤ 状态根

- 在区块头中，保存了对世界状态的一个信息摘要，也就是状态的哈希值。这个状态的哈希值在区块头中被叫作状态根，这是因为状态根的计算是通过一个特殊的树状哈希数据结构来实现的，计算出来的哈希值处于这种树状哈希结构的根。

叔块

➤ 定义

- 以太坊定义了不在主链但被主链区块记录的满足难度的区块为叔块。

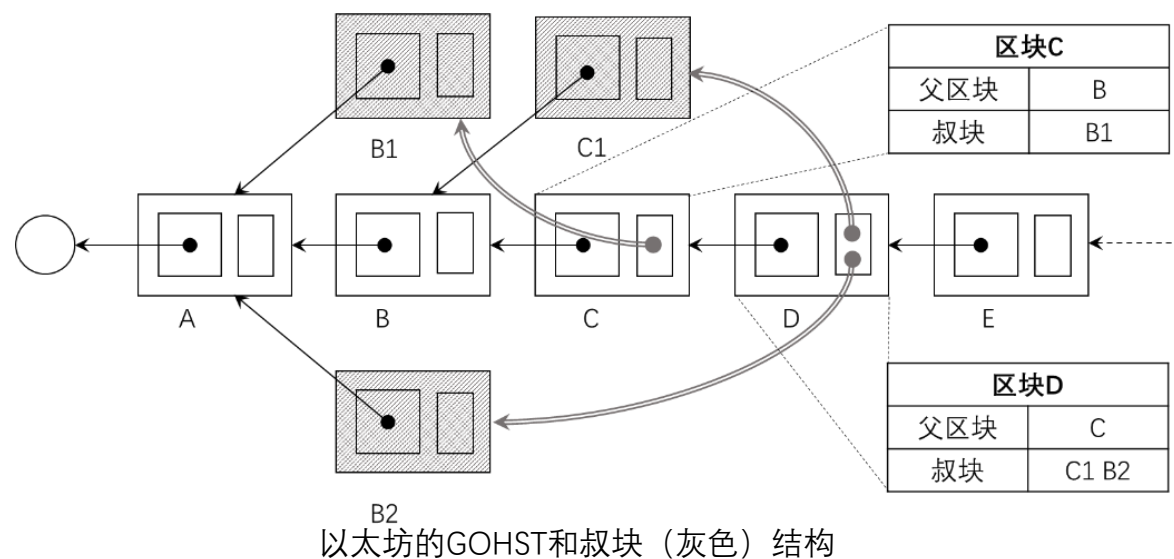
➤ 作用

- 在尽可能减少两个相邻区块产生时间的条件下，尽量收缩和统一整个区块链的主链，同时通过叔块的激励来维护矿工的积极性。

叔块

➤ 实例

- 创世区块后产生了三个分叉，其中中间的分叉被选为主链，其他两个区块可以被后继的区块当作叔块
- 被纳入主链之中，获得一定的收益补偿
- 其他矿工在打包区块的时候，可以自主的选择合法的0到2个叔块打包到新区块的区块头



收据

➤ 定义

- 收据是对应交易的数据结构，代表了交易执行的一些中间状态的写入和交易的执行结果等信息

➤ 内容

- 以太坊的智能合约向虚拟机输出的一些执行日志
- 智能合约运行的Gas信息
- 单个交易执行完毕后以太坊的状态根
- 交易创建智能合约的时候，如果执行成功会把新建合约的地址写到收据之中

Merkle Patricia Trie

➤ 背景

- 在每个区块头中，以太坊都要对全体地址和账户的哈希值进行计算，得到一个世界状态的状态根
- 直接把整个地址到账户的映射表拼接成二进制进行哈希计算的话，整个过程的开销将会十分巨大
- 单个区块中的全部交易可能仅仅改变了几十个账户中的状态，但是节点仍然要重新遍历数万甚至数千万的账户来计算状态根

Merkle Patricia Trie

➤ 比特币 Merkle Tree

- 在比特币中，Merkle Tree被用来组织交易列表计算哈希
- 对于以太坊全体账户这种映射表的结构来说，Merkle Tree的能力还是稍显不足
 - 假如使用账户地址作为列表的下标，那么潜在的列表长度是 2^{160}
 - 这个长度为 2^{160} 的列表构成的Merkle Tree将会非常巨大，总共具有 2^{161} 个节点

Merkle Patricia Trie

➤ 以太坊 Merkle Patricia Trie

- 使用16叉压缩前缀树（或者叫做基数树，Radix Tree）作为地址到账户的一个索引
- 利用Merkle Tree的思想对每一层节点合并计算所有子节点的哈希，得到根哈希值

➤ 兼具Merkle Tree和压缩前缀树的优点

- 实现有效的哈希证明
- 实现修改叶子节点数据后快速重新计算哈希值
- 对数据进行索引，能够更快地根据地址查询对应的账户

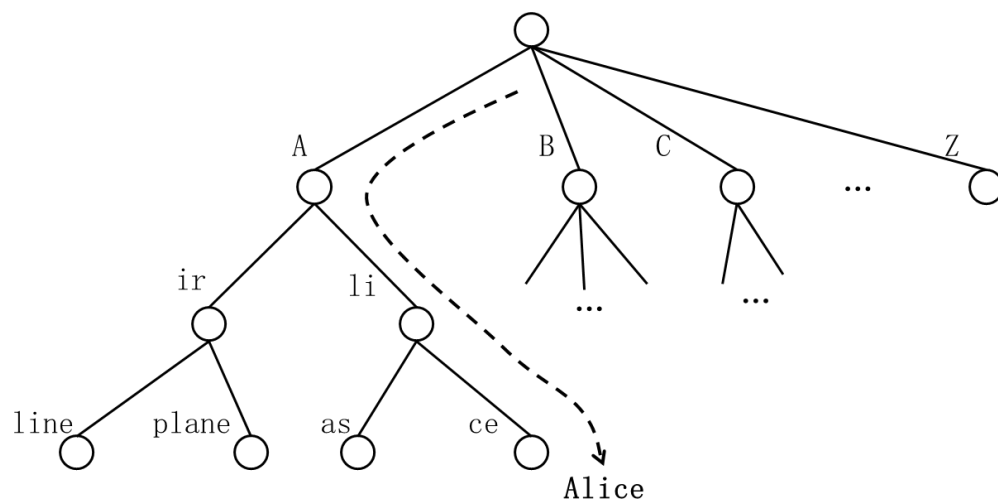
压缩前缀树

➤ 定义

- 压缩前缀树是指由公共前缀组成的一棵字典树

➤ 压缩前缀树的查找

- 如我们要在字典中搜索Alice这个单词，会先在字典中查找A开头的部分，然后进而查找Al开头的部分，最后找到单词Alice。压缩前缀树的原理也是类似地，将共同前缀的部分组成一个子树，然后随着层级向下逐层细分。



四个单词构成的压缩前缀树

MPT

➤ 背景

- 如果把单词和箭头都换成哈希值，然后计算每个中间节点的哈希值，便得到了MPT

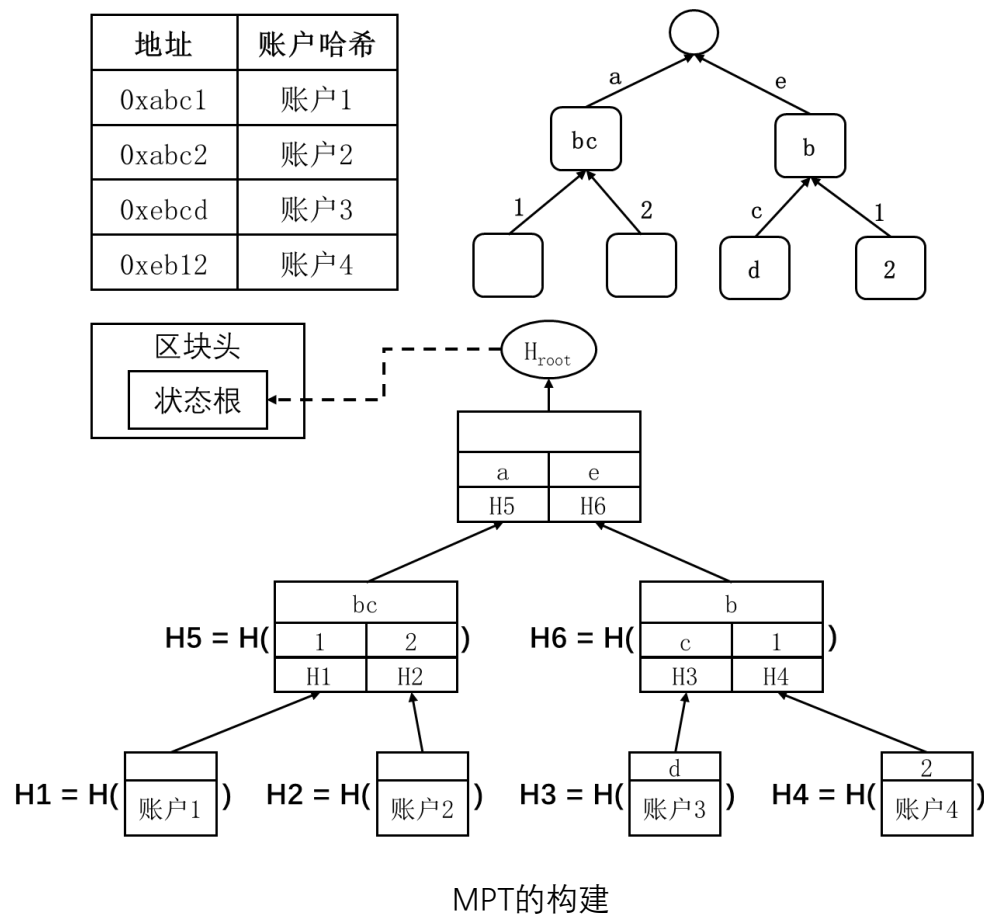
➤ MPT的构造

- 按照所有数据的地址（或者键值）来构建一棵压缩前缀树
 - 由于地址是以16进制为编码的，我们使用0123456789abcdef作为每一个编码的单元
- 按照构建得到的压缩前缀树，从叶子节点开始，逐步计算每一层的哈希值，并将其汇合到父节点中，与Merkle Tree的计算过程类似

MPT

➤ 实例：根哈希值的计算

- 计算叶子节点的哈希值
 - 空前缀加上数据1的哈希值，得到哈希值 $H1 = H("" + \text{账户1哈希})$ 。H2、H3、H4的计算类似。
- 计算父节点哈希值
 - 将前缀和子分支的哈希值一同进行哈希，得到哈希值 $H5 = H(\text{bc} + H1 + H2 + "" + \dots + "")$ ，注意其他子分支不存在，值都是空。
- 逐层计算得到根哈希值 H_{root}
 - 根哈希值 $H_{root} = H("" + "" + \dots + "" + H5 + "" + \dots + "" + H6 + "")$ ，注意到这个节点的公共前缀为空。



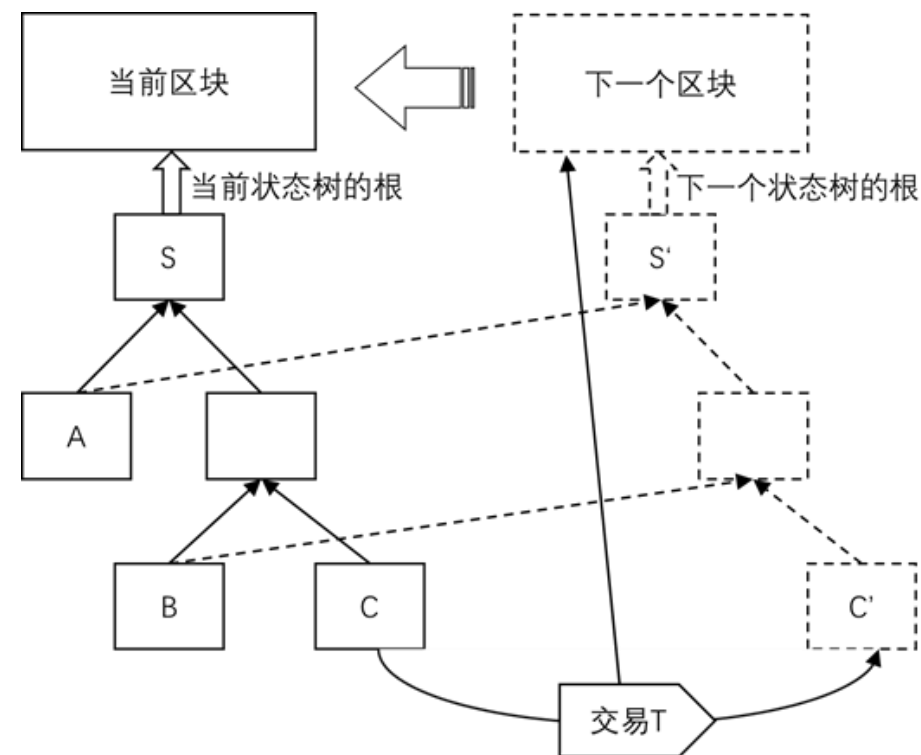
状态树

➤ 状态树

- 以太坊用来记录各个账号的状态的树，树的键是账户地址，值是账户的详细信息。

状态树

- 实例：以太坊中状态树的变换
 - 当前的区块的世界观中有状态为A、B和C三个账户
 - 经过交易T之后，账户C的状态变成了C'，计算下一个状态树的根S'
 - 以太坊将交易T和状态树根S' 记录在下一个区块之中



以太坊中状态树的变换

合约存储树

➤ 合约存储树

- 合约账户下的存储也是一个映射表，它记录了从存储地址到存储值的一个映射关系
- 在合约账户的数据结构中存储了这个映射表的哈希值，这个哈希值被称作存储根（Storage Root），它同样是由一棵MPT来维护和计算

交易树和收据树

➤ 交易树和收据树

- 与比特币中的Merkle Tree类似的，对于区块中的所有交易和交易对应的收据，都可以使用MPT进行组织和证明
- MPT的构建不再是通过账户地址来进行，而是通过交易或者收据在区块中的序号来构建MPT

布隆过滤器

➤ 用途

- 以太坊中通过布隆过滤器对收据的日志进行索引。

➤ 优点

- 布隆过滤器可以用于检索一个值是否在一个集合中。在容忍一定的误识别率的条件下，它有着远超过一般算法的空间效率和时间效率。

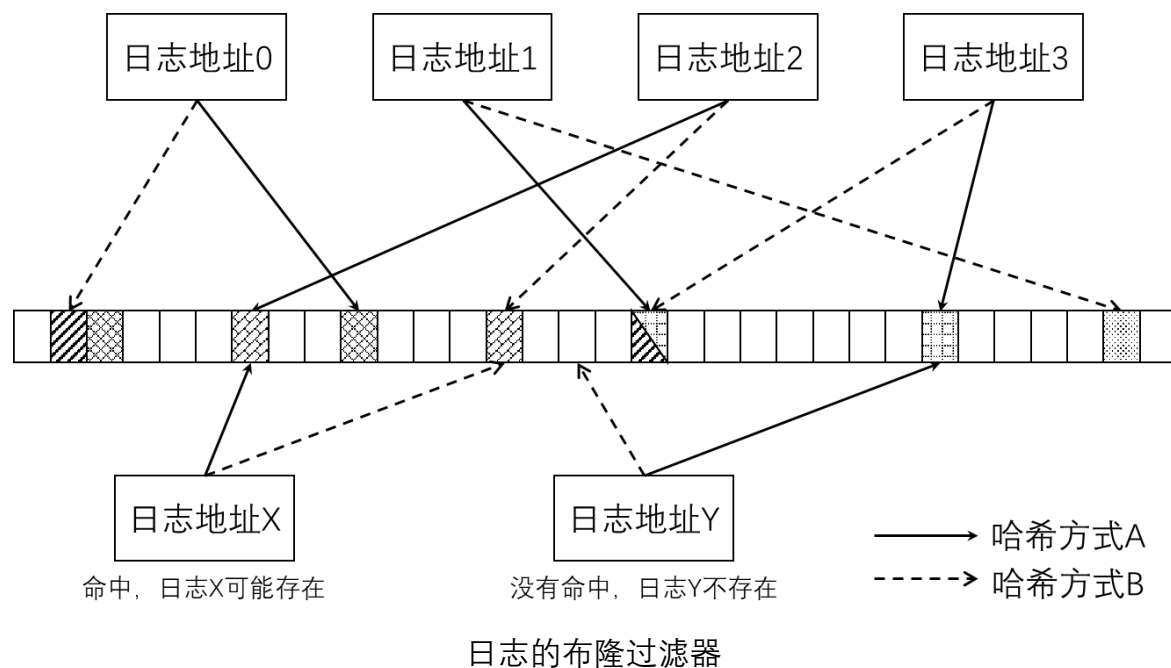
➤ 原理

- 通过多个哈希函数将键值映射到位图之中，并在位图中合并集合中所有键值的映射结果。对于一个键值，如果经过同样的哈希函数映射之后，出现了在位图中没有出现的标记位，那么这个键值必定不存在于集合之中。

布隆过滤器

➤ 实例

- 在进行哈希方式B之后，若日志地址Y的哈希出现了新的标记位，那么日志地址Y必然不存在于原有四个日志地址之中



本章总结

- 以太坊的基本架构及原理
- 以太坊账户模型与转账原理
- 智能合约运行原理
- 以太坊交易方式
- 以太坊数据结构与存储方式